

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧЕРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«Санкт-Петербургский политехнический университет Петра Великого»

ИНСТИТУТ КОМПЬЮТЕРНЫХ НАУК И ТЕХНОЛОГИЙ

Курсовой проект

Программирование на языке Ассемблера
по дисциплине «ЭВМ и периферийные устройства»

Выполнил студент группы: 23504/2:

Лаппо С.С

Руководитель
профессор, д.т.н.

Молодяков С. А.

7 декабря 2016 г.

Санкт-Петербург
2016

Содержание

1	Описание работы	2
1.1	Цели	2
2	Ход работы	3
2.1	Задача 1	3
2.2	Задача 2	7
3	Вывод	12

1 Описание работы

1.1 Цели

1. Дано арифметическое выражение. Разработать программу проверки правильности по следующим критериям:

- допустимые знаки операций (" + ", " − ", " * ", " \ ");
- допустимые константы (целые без знака);
- допустимые переменные (до пяти латинских букв);
- скобки (только круглые, они должны быть парными).

Продумать диагностику по разным типам ошибок.

2. Программа "Будильник2". Будильник должен прозвенеть в абсолютное время (11:45). Перехватите прерывание клавиатуры и при повторном наборе (нажатии клавиш). Ваша программа сообщает "Ошибка". При наступлении заданного времени выдайте звуковой сигнал.

2 Ход работы

2.1 Задача 1

Алгоритм

Для выполнения работы программа читает вводимые данные и проверяет является ли данный символ константой. Если символ является константой, то она увеличивает счётчик, отвечающий за количество символов в последней константе. Если символ является одним из знаков операций, если является то проверяет был ли предыдущий таковым. В случае подтверждения предположения - программа выдаёт ошибку и завершается, иначе счётчик отвечающий за количество символов в константе становится равным нулю. Если символ равен точке программа выдаёт ошибку.

Прерывание

int 21h

В регистре АН 01h - использовалось для чтения символа

int 21h

В регистре АН 09 - использовалось для вывода строки

int 21h

В регистре АН 4C00h - использовалось для завершения выполнений программы

Назначение

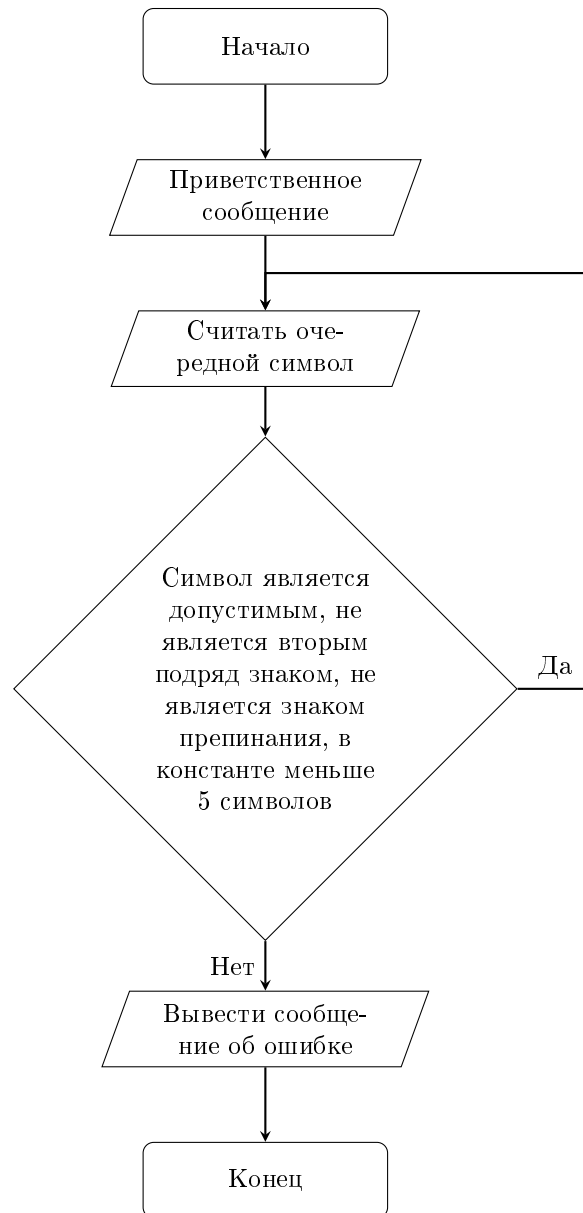
Листинг 1: Код программы

```
1 use16
2 macro exit_app
3 {
4 mov ax,4C00h
5 int 21h
6 }
7 macro print_str str
8 {
9 mov ah,9
10 mov dx,str
11 int 21h
12 }
13
14 macro read_character
15 {
16 mov ah,01h
17 int 21h
18 cmp al,13
19 je DoneOk
20 mov cl,bl
21 mov bl,al
22 compareToAll
23 je CheckPrevious
24 jne UnDone
25 }
26
27 macro compareToAll
28 {
29 cmp bl,42
30 je CheckPrevious
31 cmp bl,43
32 je CheckPrevious
33 cmp bl,47
34 je CheckPrevious
35 cmp bl,45
36 je CheckPrevious
37 }
38
39 macro compareToAllFP
40 {
41 cmp cl,42
```

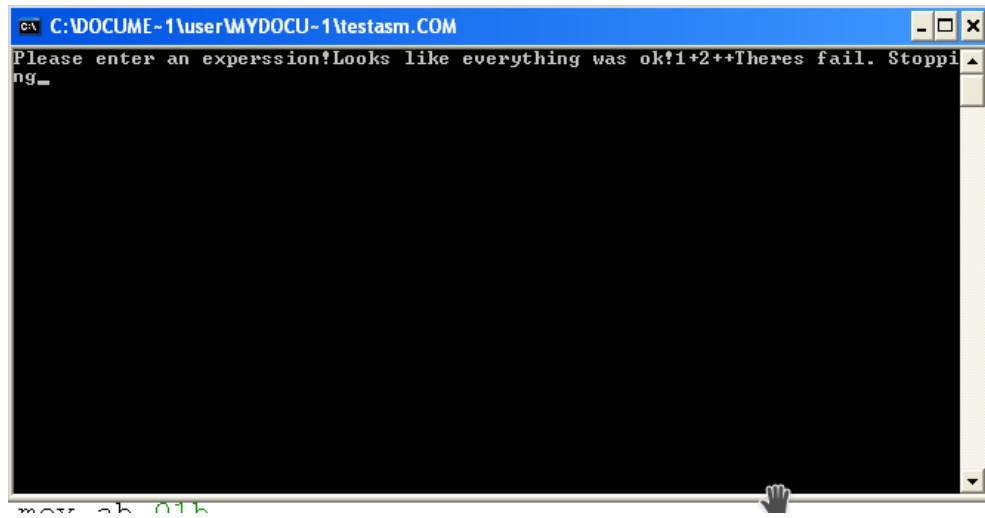
```

42  je Fail
43  cmp cl, 43
44  je Fail
45  cmp cl, 47
46  je Fail
47  cmp cl, 45
48  je Fail
49  cmp cl, 40
50  je Fail
51  }
52
53  org 100h
54  mov bx, 1
55  mov ax, 1
56  print_str hello
57
58  DoneOk:
59  print_str exit
60  read_character
61  exit_app
62
63  UnDone:
64  read_character
65
66  CheckPrevious:
67  compareToAllFP
68  read_character
69  Fail:
70  print_str failMsg
71  mov ah, 01h
72  int 21h
73  exit_app
74  ; data
75  failMsg db 'Theres fail. Stopping$'
76  hello db 'Please enter an experssion!$'
77  exit db 'Looks like everything was ok!$'

```



Скриншот выполнения:



2.2 Задача 2

Алгоритм

Для выполнения работы было решено получать системное время, преобразовывать его в строку и сравнивать с заранее заданой строкой времени будильника. При совпадении строк - подать звуковой сигнал и завершить работу программы. Так же для обеспечения работы был написан обработчик прерывания, который при нажатии клавиши увеличивает счётчик, со значением по умолчанию равным 0 на 1, и при достижении определённого значения обработчик вызывает функцию завершения программы с выводом сообщения об ошибке.

Прерывание

Назначение

int 21h	В регистре AH 2CH - использовалось для получения системного времени
int 21h	В регистре AX 3509h - использовалось для получения вектора прерываний
int 21h	В регистре AH 4CH - использовалось для завершения выполнений программы
int 21h	В регистре AX 2509h - использовалось для установки собственного обработчика прерываний
int 21h	В регистре AH 2, в регистре DL 7 - использовался для генерации звука
int 10h	Установка строки в начало (видеорежим DOS)

Листинг 2: Код программы

```
1 .MODEL LARGE
2 code segment
3 ; assume cs:code, ds:code, es:code, ss:code
4 org 100H
5
6 .DATA
7 PROMPT DB 'Current System Time is : $',0
8 TIME DB '00:00:00$',8,0 ; time format hr:min:sec
9 TIMEEST DB '11:45:00$',8,0
10 errstr DB 'ERROR$',5,0
11
12 ; Data1 SEGMENT WORD 'DATA'
13 int1 dw 0h, 0h
14 x DB 0
15 e DB ?
16 ; Data1 ends
17 .CODE
18 jmp MAIN
19
20 ;*****
21 ;_____ CONVERT _____;
22 ;*****
23
24 CONVERT PROC
25 ; this procedure will convert the given binary code into ASCII code
26 ; input : AL=binary code
27 ; output : AX=ASCII code
28
29 PUSH DX ; PUSH DX onto the STACK
30
31 MOV AH, 0 ; set AH=0
32 MOV DL, 10 ; set DL=10
33 DIV DL ; set AX=AX/DL
34 OR AX, 3030H ; convert the binary code in AX into ASCII
35
36 POP DX ; POP a value from STACK into DX
37
38 RET ; return control to the calling procedure
39 CONVERT ENDP ; end of procedure CONVERT
40
```



```

41 ;*****;
42 ;-----;
43 ;*****;
44
45
46 ;*****;
47 ;----- GET_TIME -----;
48 ;*****;
49
50 GET_TIME PROC
51 ; this procedure will get the current system time
52 ; input : BX=offset address of the string TIME
53 ; output : BX=current time
54
55 PUSH AX ; PUSH AX onto the STACK
56 PUSH CX ; PUSH CX onto the STACK
57
58 MOV AH, 2CH ; get the current system time
59 INT 21H
60
61 MOV AL, CH ; set AL=CH , CH=hours
62 CALL CONVERT ; call the procedure CONVERT
63 MOV [BX], AX ; set [BX]=hr , [BX] is pointing to hr
64 ; in the string TIME
65
66 MOV AL, CL ; set AL=CL , CL=minutes
67 CALL CONVERT ; call the procedure CONVERT
68 MOV [BX+3], AX ; set [BX+3]=min , [BX] is pointing to min
69 ; in the string TIME
70
71 MOV AL, DH ; set AL=DH , DH=seconds
72 CALL CONVERT ; call the procedure CONVERT
73 MOV [BX+6], AX ; set [BX+6]=min , [BX] is pointing to sec
74 ; in the string TIME
75
76 POP CX ; POP a value from STACK into CX
77 POP AX ; POP a value from STACK into AX
78
79 RET ; return control to the calling procedure
80 GET_TIME ENDP ; end of procedure GET_TIME
81
82
83
84
85 MAIN PROC
86 jmp beginn
87 Keyblnt:
88 cli
89 push bp
90 mov bp, sp
91 push ax
92 push bx
93 push cx
94 push dx
95 push si
96
97 push ax
98 in al,60H ;read the key
99 ; mov dx, '['

```

```

100 ; call print_symbol
101 ;
102 ; no, drop through to original driver
103 pop     ax
104
105
106 in      al,61H           ;get value of keyboard control lines
107 mov     ah,al           ; save it
108 or      al,80h          ;set the "enable kbd" bit
109 out     61H,al          ; and write it out the control port
110 xchg    ah,al           ;fetch the original control port value
111 out     61H,al          ; and write it back
112
113 mov     al,20H           ;send End-Of-Interrupt signal
114 out     20H,al          ; to the 8259 Interrupt Controller
115 ;----- other code handles other tests and finally triggers popu
116 push    dxprint_symbol
117 mov     al, x
118 add     al, 1
119 mov     x, al
120 cmp     al, 3
121 jg      exitperr
122
123 pop     dx
124 pop     si
125 pop     dx
126 pop     cx
127 pop     bx
128 pop     ax
129 pop     bp
130 sti
131 iret
132
133 beginn:
134 mov     ax, 3509h
135 int     21h
136 mov     int1, bx
137 mov     int1+2, es
138 push    cs
139 pop     ds
140 mov     dx, offset Keyblnt
141 mov     ax, 2509h
142 int     21h
143 pushf
144 pushf
145 MOV     AX, @DATA          ; initialize DS
146 MOV     DS, AX
147
148 LEA     BX, TIME           ; BX=offset address of string TIME
149
150 CALL    GET_TIME          ; call the procedure GET_TIME
151 lea     si, TIME
152 lea     di, [TIMEEST]
153 xor     dx, dx            ; make sure edx is 0 to start with
154 LOOPP:
155 mov     al, [si]
156 mov     bl, [di]
157 inc     si                ; prepare for next char
158 inc     di

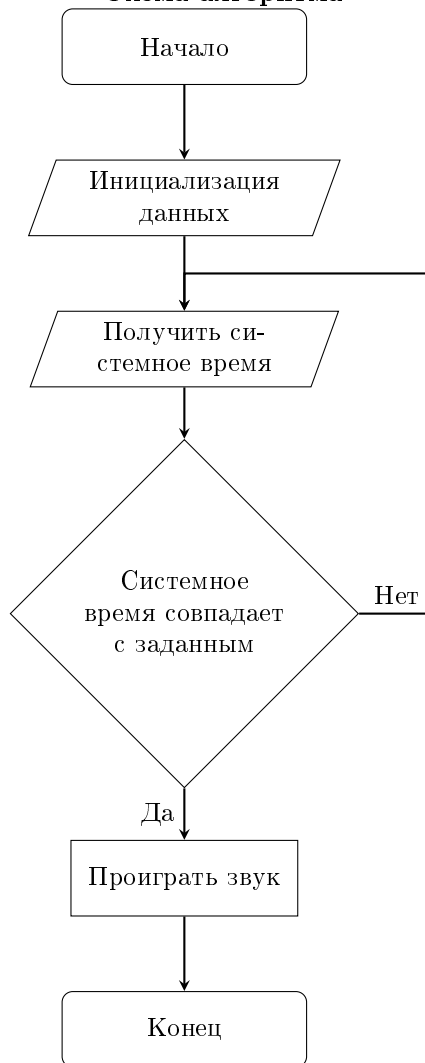
```

```

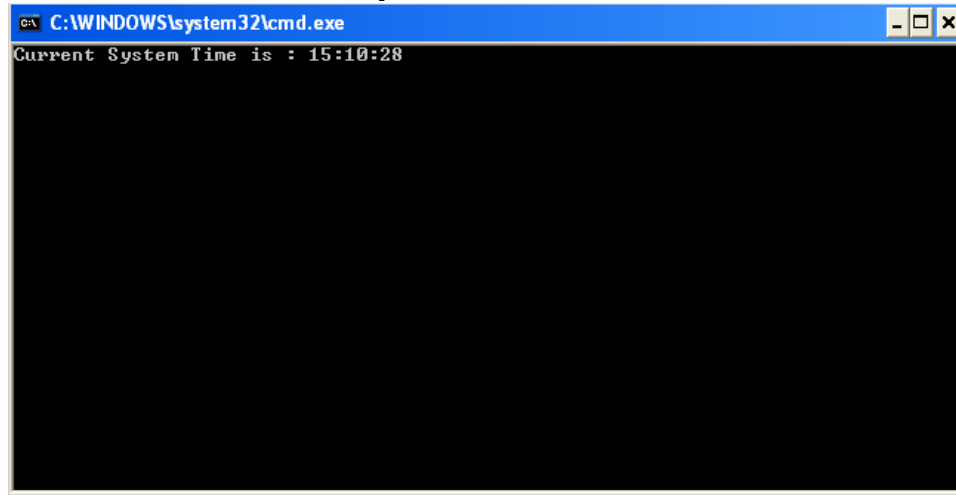
159 cmp al, bl           ; compare two current characters
160 jne DIFFERENT        ; not equal, get out, you are DONE!
161 cmp al, 0            ; equal so far, are you at the end?
162 je SAME              ; got to end of both strings, you're good, get out
163 jmp LOOPP            ; okay well they agree so far, go to next char
164 DIFFERENT:
165 ; Do what you need to do for the strings being different
166 ;
167 jmp DONE
168 SAME:
169 ; Do what you need to do for the strings being the same
170 ;
171 ;
172 jmp EXITP
173 DONE:
174 ;CMP TIME, TIMEEST
175
176 LEA DX, PROMPT        ; DX=offset address of string PROMPT
177 MOV AH, 09H          ; print the string PROMPT
178 INT 21H
179
180 LEA DX, TIME          ; DX=offset address of string TIME
181 MOV AH, 09H          ; print the string TIME
182 INT 21H
183 mov ax, 0
184 mov bp, 43690
185 mov si, 13690
186 delay2:
187 dec bp
188 nop
189 jnz delay2
190 dec si
191 cmp si, 0
192 jnz delay2
193
194 mov ah, 0
195 mov bh, 0
196
197 int 10h
198 CALL MAIN
199 EXITPERR:
200
201 mov ah, 9
202 lea dx, errstr
203 int 21h
204 MOV AH, 4CH           ; return control to DOS
205 INT 21H
206
207 EXITP:
208 mov ax, 1500
209 MOV AH, 2            ; ??????? ?????? ??????? ?? ?????
210 MOV DL, 7            ; ??????? ??? ASCII 7
211 INT 21H              ; ??????? ??????
212 MOV AH, 4CH           ; return control to DOS
213 INT 21H
214 MAIN ENDP
215 END MAIN

```

Схема алгоритма



Скриншот выполнения:



3 Вывод

В результате выполнения работы были выполнены задания, а так же улучшен навык взаимодействия с системными прерываниями,и создания собственного обработчика прерывания.